

# MS&E 228: Applied Causal Inference Powered by ML and AI

## Lecture 8: Deep Dive into OLS Coefficients and the Double Machine Learning Estimator

Vasilis Syrgkanis

Stanford University

Winter 2026

**Readings:** *Applied Causal Inference Powered by ML and AI*, §1, 4, 9.

# Goals for Today

1. **Develop deeper intuition for OLS coefficients:** What does a coefficient in a multivariate regression really mean? When is it uniquely identified, and what drives its variance?
2. **Discover another “insensitive” formula:** Can we simplify inference for OLS coefficients by ignoring nuisance estimation, similar to the DR estimator?
3. **Move beyond full linearity:** How can we allow flexible ML models for covariates while still getting valid causal inference on treatment effects?
4. **Build a practical algorithm for continuous treatments:** What estimator can we use in practice, and what does it estimate when treatment effects are heterogeneous?

# **Part I: Understanding OLS Coefficients**

The Frisch-Waugh-Lovell Theorem

# Where We Left Off

Last lecture, under full linearity  $\mathbb{E}[Y(d)|X] = \theta d + \beta'X$ , all causal estimands reduce to estimating  $\theta$ .

We posed four key questions:

1. How can we interpret  $\theta$  intuitively?
2. When is  $\theta$  uniquely identified in the best linear predictor?
3. What drives high variance in  $\hat{\theta}$ ?
4. What is the analogue of “overlap” under linearity?

Today we answer all four questions through two marvelous theorems!

# The Frisch-Waugh-Lovell Theorem

## Theorem (Frisch-Waugh-Lovell)

The coefficient  $\hat{\theta}$  on  $D$  in  $\text{OLS}(Y \sim D, X)$  is identical to the coefficient from:

**Step 1:** Regress  $Y$  on  $X$  and compute residuals—the “surprise” in  $Y$ :

$$\check{Y} = Y - \text{OLS}(Y \sim X).\text{predict}(X)$$

**Step 2:** Regress  $D$  on  $X$  and compute residuals—the “surprise” in  $D$ :

$$\check{D} = D - \text{OLS}(D \sim X).\text{predict}(X)$$

**Step 3:** Run simple line-fit on the surprises:

$$\hat{\theta} = \text{OLS}(\check{Y} \sim \check{D}) \quad \Leftrightarrow \quad \text{solve empirical normal equations } \mathbb{E}_n[(\check{Y} - \hat{\theta}\check{D})\check{D}] = 0$$

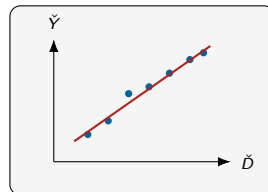
# Visual Intuition for FWL

## Original Problem

OLS( $Y \sim D, X$ )  
High-dimensional  
regression

Same  $\hat{\theta}$

## FWL Equivalent



A complex multivariate regression reduces to a simple line fit on residuals!

# The FWL Formula

From the simple line-fit OLS( $\check{Y} \sim \check{D}$ ), we get:

$$\hat{\theta} = \frac{\mathbb{E}_n[\check{Y} \cdot \check{D}]}{\mathbb{E}_n[\check{D}^2]} = \frac{\text{Cov}_n(\check{Y}, \check{D})}{\text{Var}_n(\check{D})}$$

## Key Insight

$\hat{\theta}$  is the **normalized correlation** between residualized outcome and residualized treatment—how much the “surprise” in  $Y$  moves with the “surprise” in  $D$ .

*Think of it as:* When treatment is higher than expected given confounders, is the outcome also higher than expected?

# Python Pseudocode: The FWL Theorem

```
# --- Standard OLS approach ---
model_full = LinearRegression().fit(np.c_[D, X])
theta_full = model_full.coef_[0]

# --- FWL three-step procedure ---
# 1. Residualize Y: Regress Y on X
Yres = Y - LinearRegression().fit(X, Y).predict(X)
# 2. Residualize D: Regress D on X
Dres = D - LinearRegression().fit(X, D).predict(X)
# 3. Final Regression: Regress residualized Y on residualized D
theta_fwl = LinearRegression(fit_intercept=False).fit(Dres, Yres)

# --- Verification ---
print(f"Full OLS theta: {theta_full:.6f}")
print(f"FWL OLS theta: {theta_fwl:.6f}")
```

## Poll: Interpreting OLS Coefficients

### Poll Question

What does the coefficient  $\theta$  on the first feature in  $\text{OLS}(Y \sim X)$  represent?

- A. The first coordinate of  $\mathbb{E}[XX']^{-1}\mathbb{E}[XY]$
- B. The correlation between  $Y$  and the first feature
- C. The normalized correlation between the residual of  $Y$  and the residual of the first feature, after removing linear predictions from other variables
- D. The average effect of changing the first feature by one unit



(Poll Everywhere)

# Live Coding: Verifying the FWL Theorem

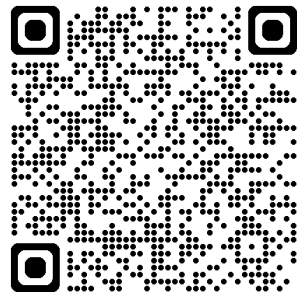
## Exercise

**Goal:** Verify that  $\text{OLS}(Y \sim D, X)$  gives the same coefficient as the FWL process.

**Steps:**

1. Run full OLS:  $Y \sim D + X$
2. Regress  $Y$  on  $X$ , get residuals  $\check{Y}$
3. Regress  $D$  on  $X$ , get residuals  $\check{D}$
4. Regress  $\check{Y}$  on  $\check{D}$
5. Compare coefficients!

**Dataset:** Price elasticity simulation



Open notebook and follow along!

# The Frisch-Waugh-Lovell Theorem (in Population Limit)

## Theorem (Frisch-Waugh-Lovell) in population

The coefficient  $\theta$  on  $D$  in the Best Linear Predictor  $\text{BLP}(Y \sim D, X)$  is identical to the coefficient from:

**Step 1:** Calculate BLP of  $Y$  using  $X$  and compute *population* residuals:

$$\tilde{Y} = Y - \text{BLP}(Y \sim X).\text{predict}(X)$$

**Step 2:** Calculate BLP of  $D$  using  $X$  and compute *population* residuals:

$$\tilde{D} = D - \text{BLP}(D \sim X).\text{predict}(X)$$

**Step 3:** Find coefficient in BLP of  $\tilde{Y}$  using  $\tilde{D}$ :

$$\theta = \text{coeff in BLP}(\tilde{Y} \sim \tilde{D}) \Leftrightarrow \text{solve normal equations } \mathbb{E}[(\tilde{Y} - \theta\tilde{D})\tilde{D}] = 0$$

## Answering Question 1: Interpreting $\theta$

**Question:** How can we better interpret the population coefficient  $\theta$  that OLS converges to?

$$\theta = \frac{\mathbb{E}[\tilde{Y}\tilde{D}]}{\mathbb{E}[\tilde{D}^2]} = \frac{\text{Cov}(\tilde{Y}, \tilde{D})}{\text{Var}(\tilde{D})}$$

### Answer

It is the slope of the line-fit between:

- ▶ The part of  $Y$  not linearly predictable from  $X$  (the “surprise” in outcome)
- ▶ The part of  $D$  not linearly predictable from  $X$  (the “surprise” in treatment)

$\theta$  captures how much the (*linearly*) unexplained variation in  $Y$  moves with the (*linearly*) unexplained variation in  $D$ , after controlling for  $X$ .

*Much more intuitive than “the first coordinate of  $\mathbb{E}[XX']^{-1}\mathbb{E}[XY]$ ”!*

# Answering Question 1: When is $\theta$ Uniquely Identified?

**Question:** When does the population BLP have a unique solution for  $\theta$ ?

## Answer

When  $D$  cannot be perfectly linearly predicted from  $X$ —i.e., when there's some “surprise” left in treatment.

- ▶ If  $D = \gamma'X$  exactly (perfect collinearity), then  $\tilde{D} = 0$  and  $\text{Var}(\tilde{D}) = 0$
- ▶ The coefficient  $\theta$  is undefined—this is the **collinearity problem**
- ▶ But this is much more benign than the overlap assumption!
- ▶ We only need  $\text{Var}(\check{D}) > 0$  **on average**, not pointwise overlap

# **Part II: Proof of the FWL Theorem**

Understanding Why It Works

# OLS as Orthogonal Decomposition

## Key Insight

Finding  $\text{OLS}(Y \sim D, X)$  coefficients is equivalent to finding a decomposition:

$$Y = \theta D + \beta' X + c + \epsilon$$

where  $\epsilon$  is empirically orthogonal to  $(1, D, X)$  (denoted as  $\epsilon \perp_n (1, D, X)$ ), i.e.,

$$\mathbb{E}_n[\epsilon] = 0, \quad \mathbb{E}_n[\epsilon D] = 0, \quad \mathbb{E}_n[\epsilon X] = 0$$

Any (coefficients, residual) pair satisfying this orthogonality is a valid  $\text{OLS}(Y \sim D, X)$  solution. *Think of it as: OLS extracts everything useful from the predictors. The residual has no remaining correlation.*

**Strategy:** We will show that the FWL procedure produces coefficients satisfying these normal equations.

# The Residualization Operator

Define the **residualization operator**: For any variable  $V$ ,

$$\check{V} = V - \gamma'_V X - c_V$$

where  $\gamma_V, c_V$  are chosen so that  $\check{V} \perp_n (1, X)$ .

*This removes the part of  $V$  that's linearly predictable from  $X$ .*

Key Property: Additivity

$$\widetilde{A + B} = \check{A} + \check{B}$$

**Proof:** Write  $A + B = (\gamma_A + \gamma_B)'X + (c_A + c_B) + (\check{A} + \check{B})$ .

Since  $\check{A} + \check{B} \perp_n (1, X)$ , this is a valid decomposition. (*The surprise in  $A + B$  is the sum of the surprises.*)

# Proving FWL

Start with OLS decomposition:  $Y = \theta D + \beta' X + c + \epsilon$ , with  $\epsilon \perp_n (1, D, X)$

**Residualize both sides:**

$$\check{Y} = \theta \check{D} + \beta' \check{X} + \check{c} + \check{\epsilon}$$

**Now observe:**

- ▶  $\check{X} = 0$  ( $X$  is perfectly linearly predictable from  $X$ )
- ▶  $\check{c} = 0$  (constant is perfectly linearly predictable)
- ▶  $\check{\epsilon} = \epsilon$  ( $\epsilon \perp_n (1, X)$  means  $\epsilon$  is its own residual)

**Therefore:**

$$\check{Y} = \theta \check{D} + \epsilon, \quad \text{with } \epsilon \perp_n \check{D}$$

Thus, the pair  $\theta, \epsilon$  is a valid OLS( $\check{Y} \sim \check{D}$ ) decomposition and hence the solution to the OLS( $\check{Y} \sim \check{D}$ ) problem! □

# **Part III: Variance and Standard Errors**

The Insensitivity Theorem

# The Insensitivity Theorem

## Theorem (Insensitivity)

When calculating standard errors for  $\hat{\theta}$ , we can ignore the estimation error in the first two OLS steps.

Asymptotically,  $\hat{\theta}$  behaves as if we had the true population residuals:

$$\tilde{D} = D - \text{BLP}(D|X), \quad \tilde{Y} = Y - \text{BLP}(Y|X)$$

*You might worry: “I estimated those residuals—doesn’t that add uncertainty?”  
Surprisingly, no!*

**Implication:** For inference, we only need to analyze a simple line-fit. Same “insensitivity” as the DR estimator!

# Deriving the Asymptotic Distribution

Starting from the population residuals:

$$\begin{aligned}\sqrt{n}(\hat{\theta} - \theta) &= \sqrt{n} \left( \frac{\mathbb{E}_n[\check{D}\check{Y}]}{\mathbb{E}_n[\check{D}^2]} - \theta \right) \approx \sqrt{n} \left( \frac{\mathbb{E}_n[\tilde{D}\tilde{Y}]}{\mathbb{E}_n[\tilde{D}^2]} - \theta \right) = \sqrt{n} \frac{\mathbb{E}_n[\tilde{D}\tilde{Y}] - \theta \mathbb{E}_n[\tilde{D}^2]}{\mathbb{E}_n[\tilde{D}^2]} \\ &= \sqrt{n} \frac{\mathbb{E}_n[\tilde{D}(\tilde{Y} - \theta \tilde{D})]}{\mathbb{E}_n[\tilde{D}^2]}\end{aligned}$$

By LLN, denominator  $\rightarrow \text{Var}(\tilde{D})$ . By CLT:

$$\boxed{\sqrt{n}(\hat{\theta} - \theta) \xrightarrow{d} N \left( 0, \frac{\mathbb{E}[\tilde{D}^2(\tilde{Y} - \theta \tilde{D})^2]}{\text{Var}(\tilde{D})^2} \right)}$$

# Computing Standard Errors in Practice

The asymptotic variance can be estimated using empirical analogues:

$$\hat{\sigma}^2 = \frac{\mathbb{E}_n[\check{D}^2(\check{Y} - \hat{\theta}\check{D})^2]}{\text{Var}_n(\check{D})^2}$$

Standard error:  $\text{s.e.} = \hat{\sigma}/\sqrt{n}$

## Practical Note

This is exactly the **HC0 robust standard error** that OLS packages compute! They don't calculate it this way, but it's numerically equivalent.

*Bottom line: just use robust standard errors from your favorite package—they're doing the right thing.*

# Python Pseudocode: Standard Errors via the Insensitivity Theorem

```
# Final stage residuals:
# outcome residual minus predicted from treatment residual
epsilon = Y_residuals - theta_fwl * D_residuals

# Asymptotic variance formula:
#  $\sigma^2 = E[(Y_{res} - \theta * D_{res})^2 * D_{res}^2] / E[D_{res}^2]^2$ 
numerator = np.mean(epsilon**2 * D_residuals**2)
denominator = np.mean(D_residuals**2)**2
sigma_sq = numerator / denominator

# Standard error = sigma / sqrt(n)
se_manual = np.sqrt(sigma_sq / len(Y))
```

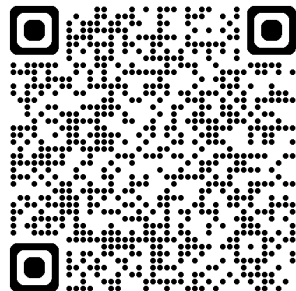
# Live Coding: Computing Standard Errors

## Exercise

**Goal:** Verify that our manual SE formula matches statsmodels HC0.

**Steps:**

1. Get SE from statsmodels (HC0)
2. Compute residuals  $\check{Y}, \check{D}$  from Part 1
3. Calculate  $\epsilon = \check{Y} - \hat{\theta}\check{D}$
4. Apply the variance formula
5. Compare:  $\text{s.e.} = \hat{\sigma}/\sqrt{n}$



Open notebook and follow along!

## Answering Question 3: Drivers of High Variance

**Question:** What are the drivers of high variance for  $\hat{\theta}$ ?

From  $\hat{\theta} = \text{Cov}_n(\check{Y}, \check{D}) / \text{Var}_n(\check{D})$ , we see two key drivers:

1. **Small denominator:** If  $\text{Var}(\check{D})$  is small (treatment is highly predictable from confounders), we divide by a small number  $\Rightarrow$  high variance  
*Little “surprise” in treatment means little to learn from.*
2. **Large numerator noise:** If the residual  $\check{Y} - \theta\check{D}$  has high variance, the estimate is noisy

### Key Insight

High linear predictability of treatment from covariates leads to high variance—but this is much weaker than requiring overlap everywhere!

## Answering Question 4: The Analogue of Overlap

**Question:** What is the analogue of “overlap” under linearity?

Answer

$$\text{Var}(\tilde{D}) > \epsilon \text{ for some } \epsilon > 0$$

- ▶ We need the residual treatment variance to be bounded away from zero
- ▶ This is an **average condition**, not a pointwise condition
- ▶ Even if  $D$  is perfectly predictable in some regions of  $X$ , we're fine as long as it's unpredictable in other regions

*Compare to binary treatment overlap ( $0 < P(D = 1|X) < 1$  everywhere)—linearity lets us extrapolate!*

## Poll 2: Variance Drivers

### Poll Question

In  $\text{OLS}(Y \sim D, X)$ , when would you expect high variance for the coefficient on  $D$ ?

- A. When  $Y$  has high variance
- B. When  $D$  can be well predicted linearly from  $X$
- C. When  $X$  has many dimensions
- D. When the sample size is small



(Poll Everywhere)

## **Part IV: Why Insensitivity Holds**

The Same Logic as Doubly Robust

# The Insensitivity Argument

Our target parameter solves:  $\mathbb{E}[(\tilde{Y} - \theta\tilde{D})\tilde{D}] = 0$

Define for arbitrary first-stage coefficients  $\gamma_Y, \gamma_D$ :

$$M(\theta, \gamma_Y, \gamma_D) = \mathbb{E}[(Y - \gamma_Y'X - \theta(D - \gamma_D'X))(D - \gamma_D'X)]$$

## Claim

This formula is **insensitive** to  $\gamma_Y$  and  $\gamma_D$  at the true values—small errors in the first stage don't affect the final answer.

*Same logic as the DR estimator!*

# Checking Insensitivity

**Partial derivative w.r.t.  $\gamma_Y$ :**

$$\partial_{\gamma_Y} M(\theta_0, \gamma_Y^*, \gamma_D^*) = -\mathbb{E}[X(D - \gamma_D^{*'} X)] = -\mathbb{E}[X\tilde{D}] = 0$$

The last equality holds because  $\tilde{D} \perp X$  by construction!

**Similarly,**  $\partial_{\gamma_D} M = 0$  by the orthogonality properties.

## Conclusion

First-stage estimation errors don't affect the asymptotic distribution of  $\hat{\theta}$ . We can use simple standard error formulas!

## Summary: Four Questions Answered

| Question                              | Answer                                                                                     |
|---------------------------------------|--------------------------------------------------------------------------------------------|
| When is $\theta$ uniquely identified? | When $D$ is not perfectly linearly predictable from $X$ (some “surprise” remains)          |
| How to interpret $\theta$ ?           | Slope of line-fit between residualized $Y$ and residualized $D$ (correlation of surprises) |
| What drives high variance?            | High noise in $Y$ , high linear predictability of $D$ from $X$ (little surprise)           |
| Analogue of overlap?                  | $\text{Var}(\tilde{D}) > \epsilon > 0$ (average condition, not pointwise)                  |

These answers come from the FWL theorem + Insensitivity theorem!

# **Part V: Beyond Full Linearity**

Partial Linearity and Double Machine Learning

# The Partial Linear Model

Full linearity is restrictive. What if we relax it?

## Partial Linear Model

$$Y = \theta D + f(X) + \epsilon, \quad \mathbb{E}[\epsilon | D, X] = 0$$

- ▶  $f(X)$  can be **any function**—no linearity assumed for how confounders affect outcome
- ▶ Only the treatment effect is assumed constant:  $\mathbb{E}[Y(d) - Y(0) | X] = \theta d$
- ▶ This is much more flexible while still enabling valid inference on  $\theta$

*We're saying: "I don't know how confounders affect the outcome—could be complicated. But each unit of treatment has the same effect."*

# The Generalized FWL Idea

What if we replace “linearly predictable” with “predictable” in FWL?

## Generalized Double Residualization:

1. Learn  $\hat{h}(X)$  predicting  $Y$  from  $X$  using **any ML method**
2. Learn  $\hat{p}(X)$  predicting  $D$  from  $X$  using **any ML method**
3. Compute residuals (surprises):  $\check{Y} = Y - \hat{h}(X)$ ,  $\check{D} = D - \hat{p}(X)$
4. Run simple line-fit:  $\hat{\theta} = \text{OLS}(\check{Y} \sim \check{D})$

**Question:** Does this recover the correct  $\theta$ ?

# Population Identification

In the population, with true conditional expectations  $h_0(X) = \mathbb{E}[Y|X]$  and  $p_0(X) = \mathbb{E}[D|X]$ :

$$\begin{aligned}\tilde{Y} &= Y - \mathbb{E}[Y|X] = \theta D + f(X) + \epsilon - \theta \mathbb{E}[D|X] - f(X) \\ &= \theta(D - \mathbb{E}[D|X]) + \epsilon = \theta \tilde{D} + \epsilon\end{aligned}$$

Since  $\mathbb{E}[\epsilon|D, X] = 0$ , we have  $\mathbb{E}[\epsilon|\tilde{D}] = 0$ .

*The  $f(X)$  terms cancel out! We don't need to know what  $f$  is.*

## Conclusion

$\theta$  is the solution to OLS( $\tilde{Y} \sim \tilde{D}$ ) in the population!

# Does Insensitivity Still Hold?

Define for arbitrary predictive models  $h, p$ :

$$M(\theta, h, p) = \mathbb{E}[(Y - h(X) - \theta(D - p(X)))(D - p(X))]$$

**Check sensitivity to  $h$ :** Let  $h_t = h_0 + t\Delta$

$$\partial_t M(\theta, h_t, p_0)|_{t=0} = -\mathbb{E}[\Delta(X)(D - p_0(X))] = -\mathbb{E}[\Delta(X)\mathbb{E}[D - p_0(X)|X]] = 0$$

**Check sensitivity to  $p$ :** Similarly equals 0.

Yes!

The formula is insensitive to the predictive models—even complex ML models!  
We can use powerful ML for confounders and still trust our standard errors.

# The Overfitting Problem

Wait!

There's a catch we saw with DR: **overfitting and double-dipping!**

If we use the same data to:

1. Train  $\hat{h}$  and  $\hat{p}$
2. Compute residuals
3. Fit the final regression

We may overfit and get biased estimates. (*The model “memorizes” training points, making residuals artificially small.*)

## Solution

**Cross-fitting**, just like with DR! For each point, predict using a model trained on *other* points.

# The Double Machine Learning Algorithm

## Algorithm (Double ML with K-fold Cross-Fitting)

1. Split data into  $K$  folds
2. For each fold  $k$ :
  - ▶ Train  $\hat{h}^{(-k)}(X)$  on out-of-fold data to predict  $Y$  from  $X$
  - ▶ Train  $\hat{p}^{(-k)}(X)$  on out-of-fold data to predict  $D$  from  $X$
  - ▶ For each sample  $i$  in fold  $k$ :  $\check{Y}_i = Y_i - \hat{h}^{(-k)}(X_i)$ ,  $\check{D}_i = D_i - \hat{p}^{(-k)}(X_i)$
3. Run simple OLS on all residuals (no intercept):

$$\hat{\theta} = \frac{\mathbb{E}_n[\check{Y}\check{D}]}{\mathbb{E}_n[\check{D}^2]}$$

# Python Pseudocode for Double ML

```
from sklearn.model_selection import KFold

def double_ml(Y, D, X, n_folds=5):
    kf = KFold(n_splits=n_folds, shuffle=True)
    Y_res, D_res = np.zeros(len(Y)), np.zeros(len(Y))

    for train, test in kf.split(X):
        # Train on out-of-fold, predict for in-fold;
        # AutoML is placeholder for any ML method
        h_model = AutoML().fit(X[train], Y[train])
        Y_res[test] = Y[test] - h_model.predict(X[test])

        p_model = AutoML().fit(X[train], D[train])
        D_res[test] = D[test] - p_model.predict(X[test])

    # Final line-fit on surprises
    theta_hat = np.mean(Y_res * D_res) / np.mean(D_res**2)
    return theta_hat
```

# Asymptotic Normality of Double ML

## Theorem

Under regularity conditions, the cross-fitted DML estimator is asymptotically normal:

$$\sqrt{n}(\hat{\theta} - \theta_0) \xrightarrow{d} N\left(0, \frac{\mathbb{E}[(\tilde{Y} - \theta\tilde{D})^2 \tilde{D}^2]}{\text{Var}(\tilde{D})^2}\right)$$

**Required conditions** (the ML models need to be “good enough”):

1. Product rate:  $\sqrt{n} \cdot \text{RMSE}(\hat{h}) \cdot \text{RMSE}(\hat{p}) \rightarrow 0$
2. Treatment model rate:  $n^{1/4} \cdot \text{RMSE}(\hat{p}) \rightarrow 0$
3. Consistency:  $\text{RMSE}(\hat{h}) \rightarrow 0$

*We saw already how Random forests, Lasso, Neural Nets typically satisfy these conditions under small “effective dimension” assumptions.*

# The Overlap Analogue in Partial Linearity

## Key Insight

The overlap condition becomes  $\text{Var}(\tilde{D}) > \epsilon$

- ▶  $\tilde{D} = D - \mathbb{E}[D|X]$  is the unpredictable part of treatment
- ▶ We need this to have positive variance **on average**
- ▶ Much weaker than pointwise overlap!

**For binary treatments:**  $\text{Var}(\tilde{D}) = \mathbb{E}[p(X)(1 - p(X))]$

- ▶ We need propensity bounded away from 0 and 1 **on average**
- ▶ Can extrapolate across covariate regions under partial linearity

# Standard Errors for Double ML

Estimate the asymptotic variance using empirical analogues:

$$\hat{\sigma}^2 = \frac{\mathbb{E}_n[(\check{Y} - \hat{\theta}\check{D})^2\check{D}^2]}{\mathbb{E}_n[\check{D}^2]^2}$$

Standard error:  $\text{s.e.} = \hat{\sigma}/\sqrt{n}$

```
def double_ml_with_se(Y, D, X, n_folds=5):  
    theta_hat, Y_res, D_res = double_ml(Y, D, X, n_folds)  
  
    # Compute standard error  
    epsilon = Y_res - theta_hat * D_res  
    sigma_sq = np.mean(epsilon**2 * D_res**2) / np.mean(D_res**2)**2  
    se = np.sqrt(sigma_sq / len(Y))  
  
    return theta_hat, se
```

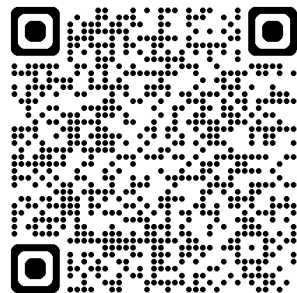
# Live Coding: Implementing Double ML

## Exercise

**Goal:** Implement DML from scratch with cross-fitting.

**Steps:**

1. Implement cross-fitting loop
2. Use FLAML AutoML to find best models
3. Compute residuals for each fold
4. Run final line-fit on residuals
5. Compute standard errors
6. Compare with true elasticity!



Open notebook and follow along!

**Key insight:** DML recovers the true effect even when  $f(X)$  is nonlinear!

# Part VII: Industry Applications

Double ML in Practice

# DML at Amazon

Amazon paper claims usage of DML in their experimentation platform for causal inference at scale.

**Paper:** “Double Machine Learning at Scale to Predict Causal Impact of Customer Actions” (Amazon Science)

## **Key applications:**

- ▶ Estimating effect of customer actions on purchases
- ▶ Price elasticity estimation
- ▶ Marketing attribution

**Why DML?** Handles high-dimensional covariates, flexible ML models, valid inference, continuous treatments, lower variance than DR, less sensitive to overlap!

# Software Implementations

## EconML (Microsoft):

```
from econml.dml import LinearDML

est = LinearDML(model_y=AutoML(), model_t=AutoML())
est.fit(Y, D, X=None, W=X)
est.summary()
```

## DoubleML (Python package):

```
from doubleml import DoubleMLPLR
from sklearn.ensemble import RandomForestRegressor

ml_g = RandomForestRegressor()
ml_m = RandomForestRegressor()
dml = DoubleMLPLR(data, ml_g, ml_m)
dml.fit()
dml.summary
```

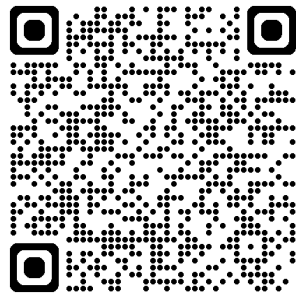
# Live Coding: Comparing with Libraries

## Exercise

**Goal:** Compare our implementation with EconML and DoubleML.

**Steps:**

1. Run EconML LinearDML
2. Run DoubleML DoubleMLPLR
3. Compare all estimates:
  - ▶ OLS (linear controls)
  - ▶ Our DML implementation
  - ▶ EconML
  - ▶ DoubleML
4. Interpret as price elasticity!



Open notebook and follow along!

## **Part VI: Interpretation with Mis-Specification**

What Does DML Estimate if CEF is not Partially Linear?

## Treatment Effect Heterogeneity: Binary Case

What if the true model has heterogeneous effects?

$$\mathbb{E}[Y|D, X] = \tau(X) \cdot D + f(X)$$

where  $\tau(X)$  varies with  $X$ . We can still run the DML procedure. Irrespective of partial linearity, DML will converge to the quantity:

$$\theta_{\text{DML}} = \frac{\mathbb{E}[\tilde{D}\tilde{Y}]}{\text{Var}(\tilde{D})}, \quad \text{with } \tilde{Y} = Y - \mathbb{E}[Y | X] \text{ and } \tilde{D} = D - \mathbb{E}[D | X]$$

### Overlap-Weighted Average Treatment Effect

DML estimates a weighted average of  $\tau(X)$ , with weights  $\propto \text{Var}(D|X)$ —more weight where treatment is more “random” (less predictable), i.e.,

$$\theta_{\text{DML}} = \frac{\mathbb{E}[\tilde{D}\tilde{Y}]}{\text{Var}(\tilde{D})} = \frac{\mathbb{E}[\tau(X)\text{Var}(D|X)]}{\mathbb{E}[\text{Var}(D|X)]}$$

# Why Use DML Even for Binary Treatments?

Even though DR works for binary treatments, DML has advantages:

1. **Lower variance:** Weights by  $\text{Var}(D|X) = p(X)(1 - p(X))$ , downweighting extreme propensities
2. **Natural interpretation:** Overlap-weighted ATE
3. **Robustness to overlap violations:** Works even with some regions of poor overlap

## Trade-off

Has lower variance and does not require overlap, but requires partial linearity assumption (constant treatment effect). If effects are heterogeneous, DML gives a weighted average, not the population ATE.

# Python Code for Binary Treatment DML

```
from sklearn.linear_model import LogisticRegression

def double_ml_binary(Y, D, X, n_folds=5):
    kf = KFold(n_splits=n_folds, shuffle=True)
    Y_res, D_res = np.zeros(len(Y)), np.zeros(len(Y))

    for train, test in kf.split(X):
        # Outcome model
        h_model = AutoML().fit(X[train], Y[train])
        Y_res[test] = Y[test] - h_model.predict(X[test])

        # Propensity model (classifier for binary D)
        p_model = AutoMLClassifier().fit(X[train], D[train])
        D_res[test] = D[test] - p_model.predict_proba(X[test])[:,1]

    theta_hat = np.mean(Y_res * D_res) / np.mean(D_res**2)
    return theta_hat
```

## Continuous Treatments: Weighted Average Derivatives

For continuous treatments **without** partial linearity, let  $g(d, X) = \mathbb{E}[Y|D = d, X]$ . Even without assuming  $g(d, X) = \theta d + f(X)$ , DML estimates a **weighted average marginal effect**:

$$\theta_{\text{DML}} = \mathbb{E} \left[ w(D, X) \cdot \frac{\partial}{\partial d} g(d, X) \Big|_{d=D} \right]$$

for some weight function  $w(D, X)$ .

*Think of it as: "On average, how much does outcome change when we nudge treatment?"*

The weights are complex in general, but DML still has a causal interpretation as a weighted average derivative!

## Special Case: Gaussian Treatment

When  $D|X \sim N(p(X), \sigma^2(X))$ , the story simplifies beautifully. The DML numerator is:

$$\mathbb{E}[\tilde{Y} \cdot \tilde{D}] = \mathbb{E}[Y \cdot \tilde{D}] = \mathbb{E}[g(D, X) \cdot (D - p(X))]$$

By **Stein's Lemma** (a result for Gaussian random variables):

$$\mathbb{E}[g(D, X) \cdot (D - p(X))] = \mathbb{E}\left[\sigma^2(X) \cdot \frac{\partial}{\partial d} g(d, X) \Big|_{d=D}\right]$$

Variance-Weighted Average Marginal Effect

$$\theta_{\text{DML}} = \frac{\mathbb{E}[\sigma^2(X) \cdot \partial_d g(D, X)]}{\mathbb{E}[\sigma^2(X)]}$$

# Interpretation of the Gaussian Case

## Key Result

Under Gaussian treatment, DML estimates the **variance-weighted average marginal effect**:

$$\theta_{\text{DML}} = \frac{\mathbb{E} [\sigma^2(X) \cdot \partial_d g(D, X)]}{\mathbb{E} [\sigma^2(X)]}$$

## Intuition:

- ▶  $\partial_d g(D, X)$  is the marginal effect at  $(D, X)$
- ▶  $\sigma^2(X)$  is the conditional variance of treatment given  $X$
- ▶ We weight by variance: more weight where treatment is more “experimental”

*Even without partial linearity, DML has a clean causal interpretation!*

## Summary: Four Key Takeaways

1. **The Frisch-Waugh-Lovell Theorem:** The coefficient  $\theta$  in  $\text{OLS}(Y \sim D, X)$  equals the slope from regressing the “surprise” in  $Y$  on the “surprise” in  $D$ —the normalized correlation of residuals.
2. **Insensitivity to First-Stage Errors:** When computing standard errors for  $\theta$ , we can ignore estimation error in the first-stage regressions—another “insensitive formula” like the DR estimator.
3. **Partial Linearity Enables Flexibility:** Assuming  $\mathbb{E}[Y|D, X] = \theta D + f(X)$  allows ML for  $f(X)$  while still getting valid inference on  $\theta$ .
4. **Double ML and Weighted Average Effects:** DML estimates variance-weighted average treatment effects (or marginal effects under Gaussian treatments via Stein’s lemma)—widely used in industry.

## Next Time

- ▶ Beyond partial linearity?
- ▶ General principle and framework of “insensitivity” for causal ML estimation problems

# References

- ▶ Chernozhukov, V., et al. (2018). Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1), C1-C68.
- ▶ Frisch, R., & Waugh, F. V. (1933). Partial time regressions as compared with individual trends. *Econometrica*, 1(4), 387-401.
- ▶ Lovell, M. C. (1963). Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis. *Journal of the American Statistical Association*, 58(304), 993-1010.
- ▶ Amazon Science. Double machine learning at scale to predict causal impact of customer actions.
- ▶ EconML Documentation: <https://econml.azurewebsites.net/>
- ▶ DoubleML Documentation: <https://docs.doubleml.org/>

# Appendix

# **Part VIII: Beyond Partial Linearity**

Flexible Dose-Response Curves

## Motivation: Nonlinear Dose-Response

Partial linearity assumes  $\mathbb{E}[Y(d)|X] = \theta d + f(X)$ —a linear dose-response.

### But what if the dose-response is nonlinear?

- ▶ Drug effects may saturate at high doses
- ▶ Price elasticity may vary with price level
- ▶ Returns to investment may be diminishing

#### Generalized Partial Linear Model

$$Y = \theta' \phi(D, X) + f(X) + \epsilon, \quad \mathbb{E}[\epsilon | D, X] = 0$$

where  $\phi(D, X)$  is a vector of basis functions (e.g., polynomials, splines). Under conditional exogeneity, this is equivalent to the assumption:

$$\mathbb{E}[Y(d) - Y(0) | X = x] = \theta' \phi(d, x)$$

# Examples of Basis Functions

**Polynomial basis:**  $\phi(D, X) = (D, D^2, D^3, \dots, D^p)'$

$$\mathbb{E}[Y(d)|X] = \theta_1 d + \theta_2 d^2 + \dots + \theta_p d^p + f(X)$$

**Spline basis:**  $\phi(D, X) = (D, (D - k_1)_+, (D - k_2)_+, \dots)'$

- ▶ Allows flexible, piecewise-linear dose-response
- ▶  $k_1, k_2, \dots$  are knot locations

**Interaction basis:**  $\phi(D, X) = (D, D \cdot X_1, D \cdot X_2, \dots)'$

- ▶ Allows treatment effect to vary with covariates
- ▶ Relaxes constant effect assumption

# DML for Generalized PLM

## Algorithm (Generalized DML)

1. Split data into  $K$  folds
2. For each fold  $k$ :
  - ▶ Train  $\hat{h}^{(-k)}(X)$  to predict  $Y$  from  $X$
  - ▶ Train  $\hat{q}^{(-k)}(X)$  to predict  $\phi(D, X)$  from  $X$  (vector-valued)
  - ▶ Compute residuals:  $\check{Y}_i = Y_i - \hat{h}^{(-k)}(X_i)$
  - ▶ Compute residuals:  $\check{\phi}_i = \phi(D_i, X_i) - \hat{q}^{(-k)}(X_i)$
3. Run multivariate OLS on residuals:

$$\hat{\theta} = (\mathbb{E}_n[\check{\phi}\check{\phi}'])^{-1} \mathbb{E}_n[\check{\phi}\check{Y}]$$

*Same idea: residualize the basis functions, then regress!*

# Python Pseudocode: Generalized DML

```
def generalized_dml(Y, D, X, phi_func, n_folds=5):  
    """phi_func(D, X) returns basis matrix of shape (n, p)"""  
    Phi = phi_func(D, X)  # (n, p) basis matrix  
    n, p = len(Y), Phi.shape[1]  
    Phi_res, Y_res = np.zeros((n, p)), np.zeros(n)  
  
    for train, test in KFold(n_splits=n_folds, shuffle=True).split(X):  
        # Outcome model  
        h_model = AutoML().fit(X[train], Y[train])  
        Y_res[test] = Y[test] - h_model.predict(X[test])  
        # Basis residuals (fit each component)  
        for j in range(p):  
            q_model = AutoML().fit(X[train], Phi[train, j])  
            Phi_res[test, j] = Phi[test, j] - q_model.predict(X[test])  
  
    # Multivariate OLS on residuals  
    theta_hat = np.linalg.solve(Phi_res.T @ Phi_res, Phi_res.T @ Y_res)  
    return theta_hat, Y_res, Phi_res
```

# DML for Generalized PLM with Separable Features

When  $\phi(d, x) = \psi(x) \cdot d$ , then residualization is easy:

$$\mathbb{E}[\phi(D, X) \mid X] = \psi(x)\mathbb{E}[D \mid X].$$

## Algorithm (Generalized DML)

1. Split data into  $K$  folds
2. For each fold  $k$ :
  - ▶ Train  $\hat{h}^{(-k)}(X)$  to predict  $Y$  from  $X$
  - ▶ Train  $\hat{p}^{(-k)}(X)$  to predict  $D$  from  $X$
  - ▶ Compute residuals:  $\check{Y}_i = Y_i - \hat{h}^{(-k)}(X_i)$
  - ▶ Compute residuals:  $\check{\phi}_i = \psi(X_i) \cdot (D_i - \hat{p}^{(-k)}(X_i))$
3. Run multivariate OLS on residuals:

$$\hat{\theta} = (\mathbb{E}_n[\check{\phi}\check{\phi}'])^{-1} \mathbb{E}_n[\check{\phi}\check{Y}]$$

*Same idea: residualize the basis functions, then regress!*

# Python Pseudocode: Generalized DML

```
def generalized_dml(Y, D, X, psi_func, n_folds=5):  
    """psi_func(X) returns basis matrix of shape (n, p)"""  
    Psi = psi_func(X) # (n, p) basis matrix  
    n, p = len(Y), Psi.shape[1]  
    D_res, Y_res = np.zeros(n), np.zeros(n)  
  
    for train, test in KFold(n_splits=n_folds, shuffle=True).split(X):  
        # Outcome model  
        h_model = AutoML().fit(X[train], Y[train])  
        Y_res[test] = Y[test] - h_model.predict(X[test])  
        # Treatment model  
        p_model = AutoML().fit(X[train], D[train])  
        D_res[test] = D[test] - p_model.predict(X[test])  
  
    Phi_res = Psi * D_res.reshape(-1, 1)  
  
    # Multivariate OLS on interacted residuals  
    theta_hat = np.linalg.solve(Phi_res.T @ Phi_res, Phi_res.T @ Y_res)  
    return theta_hat, Y_res, Phi_res
```

# Asymptotic Distribution

Under regularity conditions,  $\hat{\theta}$  is asymptotically normal:

$$\sqrt{n}(\hat{\theta} - \theta_0) \xrightarrow{d} N(0, \Sigma)$$

where  $\Sigma$  is given by the sandwich formula:

$$\Sigma = \left( \mathbb{E}[\tilde{\phi}\tilde{\phi}'] \right)^{-1} \mathbb{E}[\tilde{\phi}\tilde{\phi}'\epsilon^2] \left( \mathbb{E}[\tilde{\phi}\tilde{\phi}'] \right)^{-1}$$

**Estimated covariance matrix:** these is the HC0 robust covariance matrix that statsmodels packages calculate

$$\hat{\Sigma} = \left( \mathbb{E}_n[\check{\phi}\check{\phi}'] \right)^{-1} \mathbb{E}_n[\check{\phi}\check{\phi}'\hat{\epsilon}^2] \left( \mathbb{E}_n[\check{\phi}\check{\phi}'] \right)^{-1}$$

where  $\hat{\epsilon}_i = \check{Y}_i - \hat{\theta}'\check{\phi}_i$ .

# Python Pseudocode: Covariance Estimation

```
def generalized_dml_with_se(Y, D, X, psi_func, n_folds=5):  
    """Computes DML estimator for theta and its covariance matrix."""  
    theta_hat, Y_res, Phi_res = generalized_dml(Y, D, X, psi_func, n_folds)  
  
    # Compute residuals  
    epsilon_hat = Y_res - Phi_res @ theta_hat  
  
    # Compute HCO robust covariance matrix  
    # Formula:  $(X'X)^{-1} (X' \text{diag}(e^2) X) (X'X)^{-1}$   
    phi_t_phi_inv = np.linalg.inv(Phi_res.T @ Phi_res)  
    meat = Phi_res.T @ np.diag(epsilon_hat**2) @ Phi_res  
    cov_matrix = (phi_t_phi_inv @ meat @ phi_t_phi_inv)  
  
    # Standard errors from covariance matrix  
    se = np.sqrt(np.diag(cov_matrix) / len(Y))  
  
    return theta_hat, se, cov_matrix
```

# Inference on Conditional Dose-Response Curve / Conditional ATE

**Goal:** Construct CI for

$$\tau(d, x) = \mathbb{E}[Y(d) - Y(0) \mid X = x]$$

at specific dose  $d$  and covariate value  $x$ .

Under the Generalized Partially Linear model:

$$\tau(d, x) = \theta' \underbrace{(\phi(d, x) - \phi(0, x))}_{\delta_{d,x}}$$

Point Estimate

$$\hat{\tau}(d, x) = \hat{\theta}' \delta_{d,x}$$

## Delta Method for Conditional Dose-Response / CATE CI

Since,  $\sqrt{n}(\hat{\theta} - \theta_0) \Rightarrow N(0, \Sigma)$ , we also have that:

$$\sqrt{n}(\hat{\tau}(d, x) - \tau(d, x)) = \sqrt{n}\delta'_d(\hat{\theta} - \theta_0) \Rightarrow N(0, \delta'_{d,x}\Sigma\delta_{d,x})$$

95% CI for Conditional Dose-Response / CATE at  $d$

$$\hat{\theta}'\delta_{d,x} \pm 1.96 \cdot \sqrt{\frac{\delta'_{d,x}\hat{\Sigma}\delta_{d,x}}{n}}$$

## Inference on Average Dose-Response Curve / ATE

**Goal:** Construct CI for  $\tau(d) = \mathbb{E}[Y(d) - Y(0)]$  at a specific dose  $d$ .

Under the model:  $\tau(d) = \theta' \underbrace{(\mathbb{E}[\phi(d, X)] - \mathbb{E}[\phi(0, X)])}_{\delta_d}$

### Point Estimate

$$\hat{\tau}(d) = \hat{\theta}' \bar{\delta}_d$$

where  $\bar{\delta}_d = \mathbb{E}_n[\phi(d, X_i) - \phi(0, X_i)]$  estimates  $\delta_d$ .

*We estimate both  $\theta$  and the population average of  $\phi$ —both contribute uncertainty!*

## Delta Method for Average Dose-Response / ATE CI

**Two sources of variance:** We estimate both  $\theta$  and  $\delta_d = \mathbb{E}[\phi(d, X) - \phi(0, X)]$ .

The full variance formula:

$$\text{Var}(\hat{\tau}(d)) \approx \underbrace{\bar{\delta}_d' \hat{\Sigma} \bar{\delta}_d / n}_{\text{from estimating } \theta} + \underbrace{\text{Var}_n(\hat{\theta}' \phi(d, X_i)) / n}_{\text{from estimating } \mathbb{E}[\phi]}$$

95% CI for Average Dose-Response / ATE at  $d$

$$\hat{\theta}' \bar{\delta}_d \pm 1.96 \cdot \sqrt{\frac{\bar{\delta}_d' \hat{\Sigma} \bar{\delta}_d + \text{Var}_n(\hat{\theta}' \phi(d, X_i))}{n}}$$

*Don't forget the second term—it captures heterogeneity in  $\phi(d, X)$  across the population!*

# Python Example: Polynomial Dose-Response

```
def poly_basis(D, degree=3):
    return np.column_stack([D**k for k in range(1, degree+1)])

def poly_basis_deriv(D, degree=3):
    return np.column_stack([k * D**(k-1) for k in range(1, degree+1)])

# Fit generalized DML
theta, se, cov = generalized_dml_with_se(Y, D, X,
                                         lambda d, x: poly_basis(d, 3))

# Dose-response at d=2: include BOTH variance terms
d_eval = 2.0
delta_d = np.mean(poly_basis(d_eval, 3) - poly_basis(0, 3), axis=0)
dr_est = theta @ delta_d
var_theta_part = delta_d @ cov @ delta_d # from theta
var_phi_part = np.var(poly_basis(d_eval, 3) @ theta) # from E[phi]
dr_se = np.sqrt((var_theta_part + var_phi_part) / n)
print(f"E[Y({d_eval})] - E[Y(0)] = {dr_est:.3f} +/- {1.96*dr_se:.3f}")
```

# Inference on Conditional Average Marginal Effect

Under the Generalized Partially Linear model:

$$\text{ME}(d, x) = \theta' \underbrace{\frac{\partial \phi(d, x)}{\partial d}}_{\psi_{d,x}}$$

95% CI for Conditional Average Marginal Effect at  $d, x$

$$\hat{\theta}' \psi_{d,x} \pm 1.96 \cdot \sqrt{\frac{\psi'_{d,x} \hat{\Sigma} \psi_{d,x}}{n}}$$

# Inference on Average Marginal Effect

The AME (averaged over observed  $D, X$ ):

$$\text{AME} = \underbrace{\theta' \mathbb{E} \left[ \frac{\partial \phi(D, X)}{\partial D} \right]}_{\psi}$$

## 95% CI for Average Marginal Effect

$$\hat{\theta}' \bar{\psi} \pm 1.96 \cdot \sqrt{\frac{\bar{\psi}' \hat{\Sigma} \bar{\psi} + \text{Var}_n \left( \hat{\theta}' \frac{\partial \phi(D_i, X_i)}{\partial D} \right)}{n}}$$

where  $\bar{\psi} = \mathbb{E}_n \left[ \frac{\partial \phi(D, X)}{\partial D} \right]$  estimates  $\psi$ .